

31.10.2024г.

Изключения и with

Лекция 8

Какво е значението на:

- `--getitem--` - взимам елемент чрез индекс / ключ
- `--setitem--` - сменяме стойността на елемента чрез индекс / ключ
- `--getattr--` - когато не съществува този елемент го извикваме
- `--setattr--` - когато се задава стойност на атрибут
- `--getattribute--` - при всяко извикване на атрибут, независимо дали съществува (викаме го винаги)

Наследяване - ... Е ...

Композиция - ... ИМА ...

Изключения (Exceptions)

- Това е грешка или неочаквана аномалия, изискваща специална обработка, променяща нормалното протичане на изпълнението на програмата.

try / except - "хваща" изключението

try:
блок

except Изключение или tuple от Изключения:

except Друго Изключение:

except
хваща което и да е изключение (без BaseException)

else: # ако няма грешка се изпълнява

finally: # изпълнява се винаги

```
try:  
    me.go_outside()
```

```
except IOError as data:  
    ...
```

→ какво ще има в data зависи от изключението, но е прието да връщат годна за отпечатване стойност (str)

Когато иначе няколко try/except намираме първия блок за нашата грешка и не търсим напред по-горе.

Дефиниране на мое изключение:

- Правиш си нов клас за изключението, което ще наследява Exception

raise ... - извиква ти грешката

- Може да дефинирам класа си за собствена грешка, която да се използва по подразбиране (дефинирам си инициализатор със съобщение)

Плюси от изключенията:

- понякога са бързи
- могат да се справят с няколко проблема
- може да се справи с "неизвестни" проблеми

raise - хвърля изключението

Файлове

Можем с блокове try/except да работим с файлове

Но по-добре да използваме и with

ch

- гарантира, че файлът ще бъде затворен автоматично

with CM [as име]:

БЛОК

→ Context manager - инстанция на клас
(има имплементиран `--enter--`
`--exit--`
и `--init--`)

- изпълнява се `--enter_()` и резултата се записва в "име"
- изпълнява се "БЛОК"
- настъпило е изключение → `--exit-- (exc_type, exc_val, exc_tb)`
- няма изключение → `--exit-- (None, None, None)`

with foo() as f, bar() as b:

...

≡

with foo() as f:

with bar() as b:

...